

通过 GitHub Copilot 提供 Claude 模型



面向企业内部 AI 编码平台：从 GitHub 企业账户、Copilot 开通与 AI Credits 预算，到以 Copilot2API 为例的 /v1/messages 原生代理层，再到开发者与应用两类接入场景的身份隔离、会话保持与负载均衡设计。

REPORT DATE / 报告日期

2026-09-02

SCOPE / 范围

9 页 · 供给 / 中转 / 消费

REFERENCE / 参考实现

copilot2api

COMPANION / 姊妹文档

quick-start.html

00

OVERVIEW — 全景视图

方案介绍



PART 01 — 供给段 (1 OF 2)

01 创建企业、开通 Copilot、邀请足够的 GitHub 账户

这一段决定了整条链路的**容量上限**与**合规底线**。顺序不能颠倒：先有企业与 Enterprise Team 结构，再开通 Copilot 并放行 Claude 模型策略，最后按测算结果邀请成员并**配置 AI Credits 预算**。跳过策略开关，后面代理层会看到一个「没有 Claude 模型」的空模型列表。

STEP 01 · ENTERPRISE SETUP

建立企业与 Enterprise Team

以 **GitHub Enterprise** 为顶层容器，下挂一个或多个 **Enterprise Team**，把账户池收敛到统一的管理与审计边界内。

- 创建 **GitHub Enterprise**，在其下建立 **Enterprise Team**
- 少量账户**：建议用户自行注册 GitHub 个人账户，由 GitHub 管理员邀请加入企业并分配许可
- 多账户管理**：建议集成 **Azure Entra ID**，配置 **SSO / SCIM**
- 确定数据处理与合规诉求（数据驻留、审计日志留存）

STEP 02 · COPILOT & MODEL POLICY

开通 Copilot 并放行 Claude

Copilot 的模型可见性由**策略开关**控制，不是开通了计划就自动可用。必须在企业 / Enterprise Team 策略中显式启用 Anthropic 模型。

- 选择计划：**Copilot Business**
- 在 Copilot 策略中启用 **Anthropic Claude models**
- 确认高阶模型（Opus 级）在 Business 下的可用范围与 **AI Credits** 口径

STEP 03 · MEMBERS & CREDIT BUDGET

邀请成员并配置 AI Credits 预算

账户数由**峰值并发**决定；能烧多少高阶模型，则由 GitHub 计费里的 **AI Credits 预算**决定（测算见下一页）。

- 邀请成员加入 **Enterprise Team** 并授予 Copilot 访问权
- 在计费页按**用户 / 成本中心**配置 AI Credits 预算与告警
- 账户命名**不要模式化**：避免 `proxy-01` 类批量编号
- 账户与路由的映射关系存在管理配置里，不编码进名称

COMPLIANCE LINE

每个账户必须对应**真实的被授权使用者**。批量注册傀儡账号、多人共享一个账户、以规避额度为目的的账户池扩张，都会违反 GitHub 服务条款与可接受使用政策，且会导致账户与企业级封禁。**先与法务/采购确认授权模式，再做技术设计。**

PART 01 — 供给段 (2 OF 2)

01 账户数按并发算，额度按 AI Credits 预算配

「足够」是**两个互不替代的维度**：账户数只解决**峰值并发**，能烧多少高阶模型则由 GitHub 计费里的 **AI Credits 预算** 决定。AI Credits 在**计费实体层级共享池化**——靠多开账户「摊额度」既不成立也不合规；额度问题必须在预算与告警里解决，而不是在账户数里解决。

测算维度	口径	说明
峰值并发	$N_1 = \text{峰值并发会话} \div \text{单账户可用并发}$	单账户可承载的并发有上限；长思考 + 流式输出会长时间占用一路连接。
账户冗余	$N = \lceil N_1 \times 1.3 \rceil + 2$	覆盖账户被限流、Token 失效、成员变动等场景，保证 N 始终可用。
灰度与验证	≥ 2 个账户	预留独立账户做模型能力回归与请求头 profile 复验，不参与生产调度。
月度用量	$C = \text{月度高阶请求量} \times \text{单请求均耗 credits}$	换算成 AI Credits 口径；具体单价与计划包含量以 GitHub 计费文档 为准。
配置预算	$\text{Budget} \geq C \times 1.2$	在计费页按 用户 / 成本中心 下发预算，并设定告警阈值与超额是否硬阻断。

CHECKLIST · 上线前逐项核对

✓ Copilot 已在**企业与 Enterprise Team** 两级启用

✓ **少量账户**：用户自行注册 GitHub 个人账户，由管理员邀请并分配许可

✓ **多账户管理**：已集成 Azure Entra ID，并配置 SSO / SCIM

✓ 策略中已放行 **Anthropic Claude 模型**

✓ 目标模型在代理拉取的模型清单中**确实可见**

✓ 已按用户 / 成本中心**配置 AI Credits 预算**

✓ 预算**告警阈值与超额策略**已确定并通知到人

✗ 未使用**模式化批量账户名**

✗ 未以**多开账户**作为扩额度手段

$N = \lceil N_1 \times 1.3 \rceil + 2$

VERIFY FIRST 策略开关生效存在延迟，模型清单也会随上游调整而变化。**以代理实际拉取到的模型列表为准**做验收，而不是以后台勾选状态为准；并把「模型列表回归」与「预算消耗巡检」一起纳入例行检查。

PART 01 · CREDIT BUDGET & POLICY CHECKLIST (2 OF 2)

2026-09-02

02

PART 02 - 中转段 (1 OF 3)

反向代理层：把 Copilot 上游变成标准 API

Copilot 上游并非公开的标准 API：它有自己的鉴权流程、请求头画像与能力边界。反向代理层的职责是把这些差异**收敛成一个标准端点**。对 Claude 模型，统一走 **Anthropic 原生 /v1/messages** ——不要绕道 OpenAI / Gemini 兼容层，转换路径会丢 thinking signature、削掉 `cache_control` 等原生字段。



PART 02 — 中转段 (2 OF 3)

02 开源代理给到哪一步，企业还要补什么

Copilot2API 的设计前提是「一个 Key 固定对应一个账户」，它给到的是一个**可用的起点**而非企业级底座：账户是静态配置、用量是单机文件，更不存在账户池、负载均衡与 Session 概念。**多账户、用量记录、负载均衡、会话保持、故障转移**这五项都要企业自研——补的是调度与治理层，不是重写协议层。

能力	结论	COPILOT2API 现状（仅作起点）	企业需自行开发
多账户支持	× 需自研	<code>accounts.json</code> 静态映射 API Key 与 GitHub 账户，一对一、手工维护；每账户独立凭据与模型缓存。	与企业身份系统（SSO/IAM）打通； 账户注册表与生命周期 ；Key 的批量下发、自动轮换与吊销；账户健康探活与自动摘除。
用量记录	× 需自研	按账户 / 按模型的计数落在单机 <code>stats.json</code> ，管理界面可视化； <code>/usage</code> 仅取上游配额快照。	请求级用量账本 与集中存储； 成本分摊口径 （团队/项目/应用/Session）；额度阈值 告警与熔断 ；与 AI Credits 预算对账。
负载均衡	× 需自研	无账户池概念：一个 API Key 只会命中它绑定的那一个 GitHub 账户，不会在多个账户间分发。	账户池 + 调度器 ：基于健康度、在途并发、剩余额度与缓存亲和的加权选择；避免无状态轮询打散上下文。
会话保持	× 需自研	代理是无状态转发，不维护 Session 与账户的绑定关系。	Session → 账户 粘性映射与 TTL；粘性失效后的迁移策略；保持 prompt cache 亲和以免缓存命中率归零。
故障转移	× 需自研	上游报错原样透传给客户端，不做账户级重试或切换。	在 请求 / 轮次边界 做 failover 并记录切换原因；对连续失败账户熔断隔离与冷却恢复。

02

PART 02 — 中转段 (3 OF 3)

企业增强四件套：多账户 · 用量 · 均衡 · 粘性

四项能力互相耦合，不能各做各的：**调度依赖健康与额度数据**，**额度数据来自用量记录**，**用量记录的归属维度又来自账户与 Session 模型**。建议一次性设计好数据面，再分期实现控制面。

BUILD 01 · ACCOUNTS

多账户与账户池

把「Key ↔ 账户」的静态绑定升级为「Key ↔ 账户池」的动态选择。

- ▶ 统一**账户注册表**：凭据、认证模式、授权状态
- ▶ 凭据**隔离存储**，加密落盘，最小权限访问
- ▶ 周期性**探活**：Token 有效性、模型可见性
- ▶ 状态机：可用 / 限流 / 冷却 / 停用
- ▶ 账户与业务线的**分组与预算隔离**

BUILD 02 · METERING

用量记录与成本归因

代理自带的按账户统计只是数据源，企业需要的是可分摊的账本。

- ▶ 请求级埋点：**请求 ID、模型、耗时、结果码**
- ▶ Token 四元组：输入 / 输出 / 缓存命中 / 缓存写入
- ▶ 归属维度：**用户 · 团队 · 应用 · Session · 账户**
- ▶ 与上游 /usage 及 **AI Credits 预算**定期到账
- ▶ 阈值**告警与自动降级**（切低阶模型/限流）

BUILD 03 · BALANCING

负载均衡与调度

目标不是「平均分配请求」，而是「在不打断会话的前提下最大化可用吞吐」。

- ▶ 调度输入：健康度 · **在途并发** · 剩余额度 · 缓存亲和
- ▶ 算法：**加权最少连接** 优于简单轮询
- ▶ 新 Session 才参与选池；已有 Session 走粘性
- ▶ 限流响应（429）触发**退避与冷却**
- ▶ 预留应急账户，不参与常规调度

BUILD 04 · STICKINESS

会话保持与迁移

Claude 的 prompt cache 与工具调用状态都依赖账户连续性，跳账户 = 缓存清零。

- ▶ **一致性哈希** session_id → account
- ▶ 粘性带 **TTL**，闲置回收避免长尾占用
- ▶ 迁移只在**轮次边界**发生，并记录原因
- ▶ 迁移后主动**重建上下文**，不假设缓存仍在
- ▶ 暴露 X-Session-Id 类约定给上层应用

03 PART 03 — 消费段 · 承接 QUICK-START.HTML 结论

开发者走 1:1 隔离；应用侧设计好粘性与均衡

这是姊妹文档 `copilot2api-quick-start.html` 已经给出的核心结论，此处作为落地要求重申：**开发者工具和后端应用的风险模型不同**——前者重点是个人身份隔离与隐私边界，后者重点是账户池负载、会话连续性与故障转移。**不要用同一套轮询规则覆盖两种场景。**

DEVELOPER SCENARIO · CLAUDE CODE / OPENCODE

建议 1:1 —— 一人 · 一 Key · 一账户

- 每位开发者使用**独立 API Key**，固定映射到**唯一 GitHub 账户**；1:1 是最简形态，可用开源代理的静态映射先跑起来，但 Key 的**下发、轮换与吊销**仍要接入企业身份系统。
- 客户端统一指向代理的 `/v1/messages`（如 `ANTHROPIC_BASE_URL`），不要走兼容层。
- 禁止多人共享 Key / 账户；确保**用量归属、限额、撤销与审计**都能落到个人。
- 个人场景**不需要负载均衡**：会话天然粘在自己的账户上，缓存命中率与上下文连续性最优。
- 接入第三方 CLI 前，核对其客户端遥测与数据策略是否符合企业隐私要求。

APPLICATION SCENARIO · ACCOUNT POOL

应用侧必须自建粘性与均衡

- 应用共享账户池，**必须显式设计 Session 模型**：先定义什么是一次会话，再决定它绑定到哪个账户。
- 建立稳定的 **Session → 账户**映射，同一会话保持粘性；**按 Session 负载，而非逐请求乱跳**。
- 综合账户健康、并发、限额与缓存状态分配**新 Session**，避免简单无状态轮询。
- 故障转移尽量发生在**请求/轮次边界**并记录原因；监控每账户成功率、429/5xx、Token 用量与活跃 Session。

API Key
个人 / 应用身份



Session Router
粘性 + 健康感知



GitHub Account
独立授权与审计

04

ROLLOUT - 落地顺序与红线

按此顺序推进，每一步都可验收

01	合规先行：与法务/采购确认授权模式与账户归属，明确「账户对应真实使用者」的红线，再启动技术设计。
02	供给就位：创建企业与 Enterprise Team，开通 Copilot，在策略中放行 Claude 模型，按 $N = \lceil N_1 \times 1.3 \rceil + 2$ 邀请成员，并配置 AI Credits 预算与告警。
03	验收模型列表：以代理实际拉取到的模型清单为准，确认目标 Claude 模型可见、可调用，并留 2 个账户做回归。
04	部署代理：以 Copilot2API 起步，逐账户完成 Device Flow 认证，对内只发布 /v1/messages，打开管理面鉴权，置于企业网关之后并校准 buffering / 超时。
05	补齐调度层：按四件套顺序实现——账户池与健康探活 → 用量账本与归因 → Session 粘性 → 健康感知均衡与轮次边界 failover。
06	分场景放量：开发者走 1:1 独立 Key 先行灰度；应用侧待粘性与均衡验证通过后再接入。
07	例行巡检：模型列表回归、请求头 profile 复验、每账户成功率与 429 率、额度消耗与对账，全部纳入周期性检查。

RED LINES · 不可触碰

- × 以规避额度为目的批量注册账号
- × 多人共享同一 Key 或账户
- × 代理直接暴露在公网且无鉴权
- × 管理界面不设 Admin Token
- × 逐请求无状态轮询打散会话
- × 让 Claude 走兼容层端点造成协议降级
- × 伪造客户端画像「试探」上游能力

KEY CONFIG

- ▶ COPILOT2API_ACCOUNTS_FILE
- ▶ COPILOT2API_ADMIN_TOKEN
- ▶ COPILOT2API_AUTH_MODE